PROJECT REPORT

Low-Light Image Enhancement with Zero-DCE

LumaLift: an interactive image-enhancement application

Submitted by	Nasim Ahmed Kazi (20244111) Mujahid Kalanthar (20244510) Narendra Prajapat (20244110) Muhammed Sabab Rawoot (20244509)
Course / laboratory	Digital Signal Processing Lab — Group 5
Submitted to	_____
Department / institution	_____
Submission date	27 September 2026

A practical implementation report on a browser-based interface and a lightweight inference service for enhancing low-illumination images.

Project repository: github.com/NasimAhmedKazi/Low-Light-Image-and-Video-Enhancement-

Deployed application: https://nasimahmedkazi-solid-disco.vercel.app

1. Abstract and Project Overview

Abstract. Low-light photographs often contain insufficient visible detail, poor local contrast, and color shifts. This project presents a working image-enhancement application built around Zero-Reference Deep Curve Estimation (Zero-DCE), a learning-based approach designed to enhance low-light images without requiring paired low-light/normal-light training examples. The repository combines a browser interface, a Flask API, and a pre-trained TensorFlow Lite model executed through LiteRT. A user selects an image, previews the original, adjusts enhancement strength, submits the image for inference, compares the result with a draggable divider, and downloads the enhanced PNG.

The model is loaded lazily by the backend and reused for later requests. The API validates image data and settings, resizes input to the model's 512 × 512 inference size, invokes the TFLite model, resizes the enhanced image back to its original dimensions, and blends it with the input according to the selected strength. The same Flask application serves both the frontend and API routes, which enables a same-origin deployment on Vercel.

1.1 Problem Statement

Images captured in dim environments can be difficult to interpret or present. Conventional global brightness adjustments may lift shadows while also washing out highlights, reducing contrast, or exaggerating sensor noise. The project explores a curve-estimation model that adapts image intensity through repeated pixel-wise transformations, then makes this process accessible through a simple upload-and-preview workflow.

1.2 Objectives

- Apply a pre-trained Zero-DCE model to user-provided images.
- Provide a responsive interface for image selection, original preview, comparison, and download.
- Expose inference through a small HTTP API with validation and explicit error responses.
- Package a practical local run path and a hosted Vercel deployment.

1.3 Project Scope

The demonstrated web inference path supports still images. The interface can preview video files, and the repository also contains a separate C++ video-processing program using OpenCV, TensorFlow C++, and FFmpeg; however, the browser's Enhance action does not process video. Training scripts are included in the repository, but this report evaluates inference with the bundled model rather than claiming new training or a controlled benchmark.

2. Technical Background and Algorithm

2.1 Zero-Reference Deep Curve Estimation

Zero-DCE formulates low-light enhancement as the estimation of image-specific curves. A convolutional network predicts adjustment maps from the input image. These maps are applied iteratively to the image, allowing different pixels and regions to receive different amounts of adjustment. The method is called zero-reference because its training objective does not require paired reference images as direct supervision; it uses non-reference objectives described in the original research paper [1].

A simplified pixel update for one iteration is:

$$x_{t+1} = x_t + r_t \cdot (x_t^2 - x_t)$$

Here, x_t is the current normalized pixel value and r_t is the estimated curve value for that iteration. The trained network predicts multiple curve maps; the repository's DCENet implementation uses eight curve iterations and a final tanh activation by default. Repeated updates can lift dark pixel values while retaining spatially varying adjustment behavior.

2.2 Model in This Application

The web service uses the repository artifact Zero-DCE/model_trained/model.tflite (about 95 KB). The interpreter is created with LiteRT and two CPU threads. The endpoint sends a single RGB tensor of shape 1 × 512 × 512 × 3, normalized to the [0, 1] range. The application selects the model output named StatefulPartitionedCall:1, which the bundled model exposes as a three-channel image. This output-selection detail is necessary because the model returns additional tensors for intermediate/curve outputs.

2.3 Strength Control

The interface's 0–100% strength value is implemented as an output blend, not as a change to the learned curve maps. At 0%, the response is the unmodified input; at 100%, it is the model result; intermediate values linearly mix both images. In normalized pixel notation: $y = (1 - s)x + s\,f(x)$, where s is the strength fraction and $f(x)$ is the model output. The default setting is 72%.

This design makes the strength control easy to interpret and avoids changing the model itself. It is an application-level control; it should not be confused with a separately trained model configuration.

2.4 Preprocessing and Output

The API applies EXIF orientation correction and converts inputs to RGB. It scales to the fixed model dimensions with bilinear interpolation, runs inference, scales the output back to the original width and height, blends according to strength, clamps values to the valid display range, and encodes a PNG response.

3. System Architecture

The system is organized as a browser client, a Flask web/API layer, and a local inference runtime. The client handles file selection and original preview in the browser; the model runs on the server side of the Flask process. In the hosted configuration, Vercel packages the Flask function, frontend assets, and TFLite artifact together.

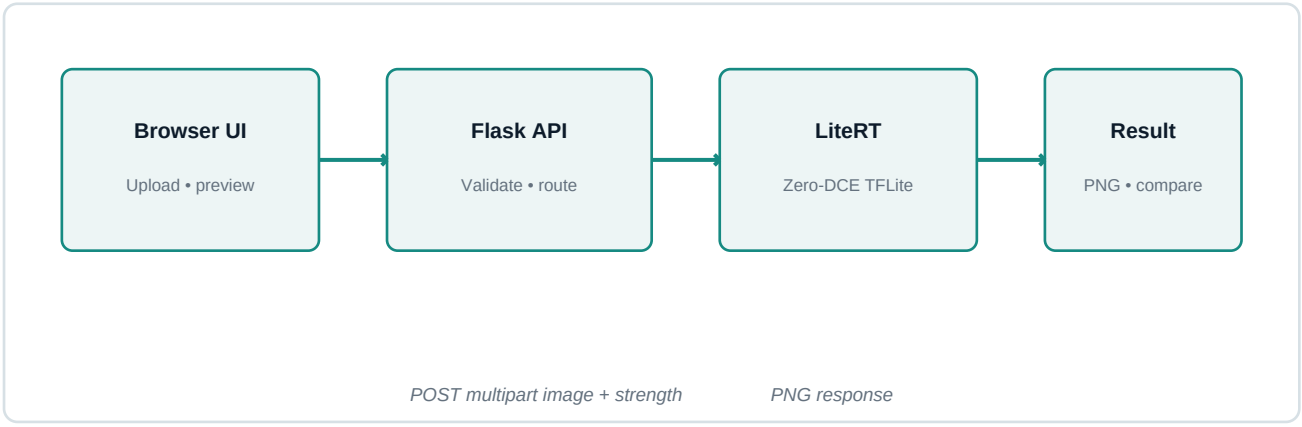

Figure 1. Image request and response path for web inference.

3.1 Main Components

- **Frontend (frontend/).** HTML, CSS, and JavaScript provide the LumaLift workspace. The browser creates a temporary object URL for the selected file, presents the original preview, submits a multipart request, displays the returned PNG, and enables download.
- **Flask application (backend/app.py).** Serves the frontend, reports model readiness through GET /api/health, and accepts images through POST /api/enhance.
- **LiteRT interpreter.** Loads the bundled TFLite file once per process, resizes the dynamic input to 512 × 512, invokes the model, and retrieves the enhanced-image output.
- **Vercel configuration.** The root app.py exports the Flask app. vercel.json includes the frontend and model files in the function bundle and sets a 60-second maximum duration. Python 3.12 uses LiteRT 2.2; local Python 3.11 uses LiteRT 1.0.1.

3.2 Request Sequence

1) The user selects an image. 2) JavaScript validates its browser-reported media type and renders the original locally. 3) On Enhance, the browser sends the file and strength as multipart form data. 4) Flask validates size, decodability, dimensions, and strength. 5) LiteRT runs inference. 6) Flask returns a PNG. 7) The browser displays the enhanced output beside the original and enables the download control.

A second, separate repository path handles video: the C++ executable reads frames through OpenCV, applies the TensorFlow C++ model, and uses FFmpeg-related audio handling. It is not part of the deployed Flask request path.

4. Implementation Details

4.1 Frontend Interaction

The page has image/video selection tabs, a drag-and-drop area, a file picker, an enhancement-strength slider, a before/after comparison surface, status messaging, and download behavior. Image selection updates the Original pane immediately using a browser object URL. Image dimensions and file name are reflected in the interface. The enhanced pane starts empty rather than showing a misleading hardcoded sample. Video files are previewable in the browser, but an attempt to enhance a video returns an explanatory message because the image model API accepts still images only.

On image enhancement, the frontend creates FormData fields named file and strength and posts them to /api/enhance. It handles JSON error responses, disables the button while processing, and ignores stale results when a newer image has been selected. Successful PNG responses are displayed via an object URL; the download action saves a file named *<original-name>-enhanced.png*.

4.2 Backend Validation

The API enforces a 25 MB request limit and returns a JSON error for oversized requests. It rejects missing files, non-numeric/out-of-range strength settings, and files that Pillow cannot decode as images. It applies EXIF transpose before RGB conversion. Images over 12 megapixels are rejected to reduce excessive memory use. Inference is guarded by a lock because the LiteRT interpreter is reused and should not receive concurrent tensor mutations.

4.3 Model Lifecycle

The model is loaded on the first health check or enhancement request and cached in process memory. This avoids reloading the small model on every request. The health endpoint returns ready=true only after the model has loaded and its input/output tensors have been inspected. A missing model or runtime issue is reported as not ready instead of showing a false success state.

4.4 Runtime Compatibility

An initial attempt to use TensorFlow directly was not reliable on the local machine because the installed TensorFlow wheel required CPU AVX instructions that were unavailable in that runtime. The repository's TFLite artifact was then tested using LiteRT, which successfully executed on CPU. This runtime choice also reduced deployment complexity: the Linux LiteRT wheel is much smaller than the TensorFlow wheel. Dependency markers select the appropriate LiteRT version for local Python 3.11 and hosted Python 3.12.

5. Testing and Results

Testing focused on the user-visible inference path and the actual model artifact. Results below describe checks performed during implementation; they are not a statistical image-quality benchmark.

Test	Observed result
Model readiness: GET /api/health	HTTP 200; {"ready": true, "model": "Zero-DCE"}
Image inference: sample dragon_inp.jpg	HTTP 200; image/png returned
Output dimensions	2068 × 1168, matching input dimensions
Output artifact	Valid 8-bit RGB PNG, about 1.30 MB
Strength validation	Invalid values rejected with HTTP 400
Invalid image validation	Undecodable input rejected with HTTP 400
Frontend flow	Selected file posted; returned blob displayed; download enabled
Production deployment	Health endpoint ready; real inference returned PNG

5.1 Sample Inference Observation

A real request to the deployed API using the repository's sample image completed successfully and returned a 2068 × 1168 PNG. The observed request took about 11.1 seconds after the model health endpoint had been called. A local request was observed at about 29.5 seconds. These are single-run observations on different environments, not controlled timing measurements; actual latency depends on cold starts, available CPU, image dimensions, and platform load.

5.2 Interpretation

The tests establish that the application can accept an image, invoke the bundled learned model, and return a displayable output with original dimensions. They do not establish a universal perceptual improvement score. A rigorous quality evaluation would require a representative test set, defined baselines, and objective metrics such as NIQE or LOE, alongside visual inspection; no such aggregate results are claimed here.

6. Deployment and User Guide

6.1 Hosted Application

The production application is available at https://nasimahmedkazi-solid-disco.vercel.app. The Vercel deployment packages the Flask entrypoint, browser assets, and TFLite model in the same function. The deployment health endpoint and an actual image enhancement request were verified after publication. Inference is subject to the configured 60-second function duration.

6.2 Local Setup

From the repository root, use Python 3.11 and run:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install -r backend/requirements.txt
python backend/app.py
```

Open http://localhost:5001. The Flask app serves the UI and endpoints from the same origin. Keep the process open while using the application. The first inference initializes the model, so it can take longer than subsequent requests.

6.3 User Procedure

- Open the deployed URL or local application.
- Choose the Image tab and select or drop a supported image.
- Confirm the original preview and displayed image dimensions.
- Set enhancement strength; use 72% as the default starting point.
- Click Enhance and wait for the model response.
- Drag the comparison divider to inspect original and enhanced areas.
- Click Download to save the enhanced PNG.

For a video, the browser provides preview only. The web enhancement button intentionally reports that video enhancement is unsupported through this image inference endpoint. The repository's separate C++ workflow is intended for video processing and requires its own native dependencies.

6.4 Repository and Deployment Configuration

The Flask app is exported from the root app.py for Vercel's Python runtime. requirements.txt selects Flask, Pillow, NumPy, and LiteRT. The .python-version file selects Python 3.12 for Vercel; backend/requirements.txt uses environment markers so local Python 3.11 installs LiteRT 1.0.1 while Python 3.12 installs LiteRT 2.2. vercel.json includes only frontend assets and the model file in the function bundle.

7. Limitations, Future Work, and Conclusion

7.1 Current Limitations

- Only still-image inference is available through the web interface; video enhancement is not wired to the Flask API.
- Input is resized to 512 × 512 for model execution before resizing back. This can soften fine details and alter aspect ratio during inference; the displayed/downloaded result is restored to original dimensions but is not a native-resolution model pass.
- The API caps uploads at 25 MB and 12 megapixels. Larger images require tiling or a higher resource budget.
- The 0–100 strength setting blends the model output with the original; it does not re-run model inference at different curve parameters.
- Single-request latency varies and can approach the serverless function time limit on slower or cold instances.
- The bundled model is small and convenient for demonstration, but this report does not establish that it is the best-performing model for every scene or camera.
- The repository identifies the Zero-DCE paper/model use as non-commercial; confirm applicable license terms before reuse outside an academic demonstration.

7.2 Recommended Next Steps

- Add a video-processing API with frame batching, progress reporting, audio preservation, and output-size limits.
- Evaluate a diverse image set and report visual comparisons plus defined no-reference image-quality metrics.
- Add automated browser/API tests to CI and retain regression images for model-output checks.
- Consider aspect-ratio-preserving padding, tiling, or configurable model resolution for high-resolution input.
- Move model inference to a dedicated GPU/CPU service if traffic or latency exceeds serverless limits.

7.3 Conclusion

This project turns a research-based low-light image enhancement model into an end-to-end application. The browser lets users select an image and inspect an original preview; the Flask service validates the request, runs the bundled Zero-DCE TFLite model through LiteRT, and returns an enhanced PNG for comparison and download. The model endpoint was verified both locally and in the production Vercel deployment. The implementation demonstrates a practical deployment path while clearly identifying still-image scope, resource limits, and the difference between a verified inference flow and a full video enhancement product.

References

[1] C. Guo, C. Li, J. Guo, C. C. Loy, J. Hou, S. Kwong, and R. Cong, “Zero-Reference Deep Curve Estimation for Low-Light Image Enhancement,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 1780–1789. doi: 10.1109/CVPR42600.2020.00185. https://openaccess.thecvf.com/content_CVPR_2020/papers/Guo_Zero-Reference_Deep_Curve_Estimation_for_Low-Light_Image_Enhancement_CVPR_2020_paper.pdf

[2] Project source repository, “Low-Light-Image-and-Video-Enhancement-.” https://github.com/NasimAhmedKazi/Low-Light-Image-and-Video-Enhancement-

[3] Google AI Edge, “LiteRT / TensorFlow Lite Interpreter,” https://ai.google.dev/edge/litert

[4] Vercel, “Python Runtime and Functions,” https://vercel.com/docs/functions/runtimes/python